

CS F407: Bayesian Networks and Autoregressive Models

AI Laboratory

Model and conditional probabilities

The six supplied sentences are lowercased and split into word tokens. Training adds START and END; order two uses two initial START tokens. First-order counts estimate $\hat{P}(v | w) = C(w, v) / \sum_u C(w, u)$. Order two uses the previous two words as its context. No machine-learning library, pretrained model, or smoothing is used.

Current word	Nonzero next-word probabilities	Greedy next
the	cat: 1/4; dog: 1/4; mat, park, rug: 1/6 each	cat
cat	sat: 2/3; ran: 1/3	sat
dog	sat: 2/3; ran: 1/3	sat
sat	on: 1	on
ran	to: 1	to

All unlisted outcomes are zero. The table uses actual dataset counts, not the handout's illustrative example counts. Observed rows sum to one within 10^{-12} . Manual checks cover these five contexts. An unseen context yields no prediction and stops generation. Greedy ties use alphabetical token order.

Generation and model comparison

Sampling uses `random.Random(7)` and stops at END. A 40-word cap prevents endless runs and is recorded separately from END. Twenty samples per order are in `generated_order1.txt` and `generated_order2.txt`; five greedy outputs and the first five samples are in `results.json`.

Measure	Order 1	Order 2
Observed contexts	11	15
Nonzero CPT entries	17	19
Free parameters on observed support	6	4
Candidate contexts	11	121
Unseen candidate contexts	0	106
Zero entries in observed rows	104	146
Dense CPT entries	121	1331
Dense CPT free parameters	110	1210
Unique sentences / 20 samples	13	6
Samples reaching END / 20	20	20

Candidate counts use raw Cartesian tables over 11 conditioning tokens and 11 outcomes. Some order-two histories contain START in an impossible position; the 106 unseen contexts include those. Free parameters on observed support equal nonzero entries minus observed rows, rather than the full dense model dimension. All five first-order greedy runs repeat *the cat sat on* and stop at the length cap. All five order-two greedy runs generate *the cat sat on the mat* and reach END. The first sampled first-order output is:

the cat sat on the dog ran to the dog ran to the park

It exposes the loss of context. All six unique order-two samples are training sentences. More local consistency coincides with less diversity on this dataset; this is not a general fluency evaluation.

Answers to Questions 1–14

1. The chain rule gives successive next-token distributions for sequential generation.
2. $P(X_t | X_1, \dots, X_{t-1}) = P(X_t | X_{t-1})$: given the previous token, earlier tokens supply no further information.
3. The five requested CPT rows are above; unlisted transitions are zero.
4. `LanguageModel.counts` stores context tuples and counters.
5. `distribution` divides each transition count by its row total.
6. `predict` chooses an argmax. Weighted `random.choices` samples any outcome with positive probability.
7. Unknown contexts give no distribution or prediction; generation reports `unknown_context` without inventing a transition.
8. A total of 0.87 indicates an incomplete or incorrectly normalised row.

9. Counts need not match human expectations: first-order prediction of *cat* after *the* ignores a preceding *on*.
10. Greedy outputs coincide; sampling varies. First-order greedy generation also loops, so its outputs are marked as truncated.
11. Order two adds a second parent, tuple contexts, and triple counts. Its larger CPT uses more context and requires more data.
12. More context disambiguates predictions but spreads the data over more rows, leaving many unobserved.
13. A behavioural prompt fixes the model and its tests. A broad request could substitute a neural model or omit probabilistic sampling.
14. The BN view supplies dependencies, a joint factorisation, independence assumptions, conditional generation, and validation invariants. Neural language models retain the next-token factorisation while learning distributions over larger contexts.

LLM use. An LLM supplied counting and generation code. Hand calculations checked counts and every generated transition was replayed. The length cap exposes greedy cycles; truncated outputs are not claimed as completed sentences.

Reproduce. Run `../.venv/bin/python solution.py` here. Results: `results.json`; prompt and correction record: `prompts.txt`.